

# **NÚCLEO DE PESQUISA: ENGENHARIA DE SOFTWARE**

## **ASSISTIDA POR INTELIGÊNCIA ARTIFICIAL GENERATIVA**

---

Esta pesquisa acadêmica consolida o estado da arte sobre a aplicação de Inteligência Artificial Generativa no Ciclo de Vida de Desenvolvimento de Software (SDLC), estruturando a fundamentação teórica necessária para o Trabalho de Graduação.

### **A. DELIMITAÇÃO**

A Engenharia de Software Assistida por IA Generativa (GenAI-SE) estende-se além do paradigma de 'autocompletar sintático', abrangendo a inserção de Modelos de Linguagem de Larga Escala (LLMs) em todas as fases do Ciclo de Vida de Desenvolvimento de Software (SDLC) — englobando levantamento de requisitos, arquitetura, geração e transformação de código, testes automatizados e refatoração. O foco recai sobre a transição dos níveis de autonomia (da assistência passiva à execução de agentes autônomos) e os impactos dessa adoção sobre a produtividade, qualidade arquitetural, e as métricas de dívida técnica e manutenibilidade a longo prazo.

### **B. PERGUNTAS DE PESQUISA**

1. Como a IA generativa se diferencia operando como ferramenta auxiliar, geradora de artefatos integrados e agente de programação autônomo ao longo do SDLC?
2. Quais são as evidências empíricas sobre os efeitos paradoxais da adoção da IA na produtividade dos desenvolvedores versus a compreensão e qualidade do código mantido?
3. Como a IA generativa está sendo incorporada à Engenharia de Software e quais condições processuais e de governança precisam ser estabelecidas para que sua utilização seja controlada, verificável e efetiva?

### **C. SUBTEMAS INVESTIGADOS**

- Diferenciação tecnológica: IA assistiva vs. geração de artefatos vs. agentes autônomos de programação (Agentic SE).

- Aplicação no SDLC: Requisitos, arquitetura, codificação, geração de testes, debugging e refatoração.
- Impactos quantitativos e qualitativos: Efeitos sobre produtividade, qualidade do código, manutenibilidade e a compreensão sistêmica.
- Colaboração Humano-IA: Níveis de autonomia, supervisão interativa, revisão de código e transferência de carga cognitiva.
- Riscos mapeados: Vulnerabilidades de segurança, alucinações de lógica de negócios e erosão do conhecimento (lock-in de raciocínio).

## **D. ESTRATÉGIA E CRITÉRIOS DE SELEÇÃO**

A busca sistemática concentrou-se na literatura de alto impacto publicada entre 2023 e 2026, com foco em anais de conferências internacionais da IEEE e ACM (ICSE, FSE) e repositórios acadêmicos (arXiv). Foram priorizados estudos empíricos envolvendo desenvolvedores humanos lidando com código real, Revisões Sistemáticas da Literatura (SLRs) e propostas de frameworks de agentes autônomos (como SWE-agent). Na esfera nacional, buscou-se a produção dos anais do Simpósio Brasileiro de Engenharia de Software (SBES). Artigos opinativos não revisados por pares foram descartados.

## **E. FONTES E F. EVIDÊNCIAS EXTRAÍDAS**

### **Fonte 1**

Autor(es): HOU, X. et al.

Ano: 2023

Título: Large Language Models for Software Engineering: A Systematic Literature Review

Periódico: ACM Transactions on Software Engineering and Methodology (TOSEM) / arXiv:2308.10620

Tipo: Revisão Sistemática (SLR).

Evidências: Sustenta a classificação do uso de IA em todo o SDLC. A fonte mapeia empiricamente que a geração de código e a geração de testes são as áreas de maior

sucesso, enquanto a etapa de levantamento de requisitos sofre com alta sensibilidade ao prompt. Evidencia o fenômeno das alucinações lógicas (Seção 4 e 6).

## **Fonte 2**

Autor(es): YANG, J. et al.

Ano: 2024

Título: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

Periódico: arXiv:2405.15793

Tipo: Estudo Empírico e Arquitetura de Software.

Evidências: Consolida a definição de "IA como Agente". A fonte demonstra que modelos encapsulados em Interfaces Agente-Computador (ACI) podem interagir autonomamente com terminais, instalar dependências de sistema e depurar (debug) código através de iteração em sandbox, resolvendo \*issues\* reais do GitHub sem intervenção humana a cada linha. (Abstract e Metodologia).

## **Fonte 3**

Autor(es): VAITHILINGAM, P.; ZHANG, T.; GLASSMAN, E. L.

Ano: 2022

Título: Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models

Periódico: CHI Conference on Human Factors in Computing Systems. DOI: 10.1145/3491101.3519665

Tipo: Estudo Empírico de IHC e Engenharia.

Evidências: Demonstra os efeitos paradoxais sobre a produtividade. A IA reduz o tempo de digitação (boilerplate), mas impõe um atraso substancial na etapa de debugging, exigindo que o desenvolvedor transite de "escritor" para "revisor e auditor de código". Revela que códigos gerados possuem maior taxa de falhas lógicas silenciosas.

## **Fonte 4**

Autor(es): BRANCO, B. M. C. et al.

Ano: 2025

Título: Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência

Periódico: Anais do Simpósio Brasileiro de Engenharia de Software (SBES). p. 470-481.

Tipo: Relato de Experiência e Estudo Educacional.

Evidências: Confirma a efetividade da IA na geração de documentação e estruturação de requisitos no cenário brasileiro, mas aponta os riscos da "erosão do conhecimento" caso os desenvolvedores iniciantes confiem cegamente na saída dos modelos para estruturar projetos.

## **G. SÍNTESE CRÍTICA**

A literatura diferencia o avanço da IA em três camadas claras:

1. Ferramenta Auxiliar (Chatbot/Oráculo): A IA não toca no código-fonte do projeto. É utilizada para consulta de sintaxe, explicação de Design Patterns ou resolução de dúvidas algorítmicas pontuais. A autonomia é zero e a colaboração é passiva.
2. Geradora de Artefatos (Copilot): Caracteriza o desenvolvimento parcialmente automatizado. A IA é embarcada no IDE e analisa o contexto do arquivo para injetar código, docstrings ou testes em tempo real. Exige supervisão humana síncrona, pois gera sugestões que o humano aceita (Tab) ou rejeita.
3. IA como Agente (Agentic SE): Caracteriza o desenvolvimento autônomo. O agente recebe uma issue abstrata (ex: "corrija o erro X de roteamento"), clona o repositório em um contêiner, edita arquivos, roda o compilador e submete um Pull Request. A colaboração humana torna-se estritamente avaliativa (Code Review assíncrono).

## **H. COMPOSIÇÃO ACADÊMICA ÚNICA**

A incorporação da Inteligência Artificial Generativa na Engenharia de Software representa uma alteração fundamental na ontologia do trabalho do desenvolvedor. A literatura recente (HOU et al., 2023; YANG et al., 2024) evidencia que a tecnologia deixou de ser uma ferramenta puramente passiva de resolução de dúvidas (ferramenta auxiliar) para atuar como uma geradora proativa de artefatos ao longo de todo o Ciclo de Vida do Software (SDLC) e, mais recentemente, como agentes dotados de autonomia encapsulada.

Os efeitos dessa integração apresentam uma natureza paradoxal. No tocante à produtividade, o desenvolvimento parcialmente automatizado via copilotos de código acelera maciçamente as fases de codificação trivial, estruturação de esqueleto de classes (boilerplate) e geração de rotinas repetitivas. Contudo, estudos empíricos rigorosos, como os conduzidos por Vaithilingam et al. (2022), revelam que essa velocidade na digitação cobra um pedágio severo nas etapas subsequentes: a depuração (debugging) e a manutenção. Como a IA não 'compreende' deterministicamente as regras de negócio intrínsecas, ela frequentemente fabrica algoritmos sintaticamente críveis, mas que violam invariantes da arquitetura e introduzem falhas lógicas.

Essa realidade exige uma reconfiguração na carga cognitiva: o engenheiro humano deixa de ser primariamente um 'escritor de sintaxe' e torna-se um 'auditor de lógica e revisor de segurança'. Sem esse deslocamento metodológico, a adoção da IA gera a erosão do conhecimento arquitetural — fenômeno no qual os mantenedores do software perdem a compreensão do porquê o sistema foi desenhado daquela maneira, prejudicando a manutenibilidade a longo prazo (BRANCO et al., 2025).

Portanto, para que a utilização da IA generativa seja controlada, verificável e efetiva, a engenharia de software contemporânea demanda a adoção de metodologias de "supervisão por design". É mandatório estabelecer políticas de Code Review humano inflexíveis para qualquer código gerado por máquina. A automação deve ser contrabalançada com rigorosos pipelines de Integração Contínua (CI/CD) onde análises estáticas de segurança e suítes exaustivas de testes unitários (os quais também podem ser gerados de forma assistida) atuem como guardiões da corretude do artefato antes de sua aprovação para os ambientes de produção.

## **I. IMPLICAÇÕES METODOLÓGICAS PARA A TG**

Para o Trabalho de Graduação focado na plataforma de gestão da grade horária da FATEC Barueri, a literatura impõe decisões metodológicas estritas. Ao codificar o sistema — especialmente em sua infraestrutura backend utilizando Java 21, Spring Boot e persistência relacional com MySQL — a equipe deve adotar um modelo de desenvolvimento parcialmente automatizado sob estrita supervisão (nível Copilot).

A metodologia do grupo deverá incorporar:

1. Micro-Engenharia de Requisitos e Prompts: As regras de alocação de turmas, choque de horários e diretrizes do SIGA deverão ser atomizadas antes de serem passadas à IA. A equipe não deve pedir para a IA 'criar a gestão de horários', mas sim focar na geração assistida de mapeamentos específicos do Hibernate ou controladores REST menores no Spring Boot.

2. Revisão por Pares (Pull Requests): Qualquer código ou query MySQL elaborada pela IA deverá passar obrigatoriamente por Code Review assíncrono por outro membro do grupo. Isso previne alucinações arquiteturais e garante que todos os membros compreendam o ecossistema Java gerado.

3. Automação Focada em Testes: O uso mais seguro da IA neste projeto será na Geração de Testes Unitários. O grupo deve utilizar a ferramenta para criar testes abrangentes sobre a lógica de horário, usando os testes automatizados como o mecanismo 'verificável e efetivo' que atesta a sanidade do código antes dos deploys.

## **J. REFERÊNCIAS ABNT COMPLETAS**

BRANCO, B. M. C.; MELO NETO, A. L. de; AVELINO, G. A.; MACHADO, V. P. Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência. Em: Anais do Simpósio Brasileiro de Engenharia de Software (SBES). Recife: SBC, 2025. p. 470-481. Disponível em: <https://sol.sbc.org.br/index.php/sbes/article/view/37023>. Acesso em: 6 out. 2026.

HOU, X. et al. Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology (TOSEM), Nova York, v. preprint, 2023. Disponível em: <https://arxiv.org/abs/2308.10620>. Acesso em: 6 out. 2026.

VAITHILINGAM, P.; ZHANG, T.; GLASSMAN, E. L. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models.

Em: CHI Conference on Human Factors in Computing Systems. Nova York: ACM, 2022.  
DOI: 10.1145/3491101.3519665.

YANG, J. et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv, [S. l.], maio 2024. Identificador: arXiv:2405.15793. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 6 out. 2026.

## **K. LACUNAS DE INVESTIGAÇÃO**

- A literatura ainda não apresenta diretrizes metodológicas unificadas (equivalentes ao Scrum ou Kanban) desenvolvidas nativamente para times compostos por humanos e Agentes de IA.
- Existem lacunas empíricas substanciais sobre o impacto de longo prazo do uso intensivo de LLMs na formação da lógica de programação de desenvolvedores novatos, e como avaliar se a curva de aprendizado de frameworks corporativos está sendo acelerada ou terceirizada para a ferramenta.